

El modelo de datos de Residente y Estado Basal

Por qué se llama "Baseline", qué entidades lo forman y una propuesta para nombrarlo en español

ResidApp — documento de arquitectura

Preparado para CJ (Product Owner) y para cualquier desarrollador que se incorpore al proyecto

Septiembre de 2026

Índice

1. Por qué llamamos "Baseline"
2. Cómo está organizado el modelo: seis bloques funcionales
3. Diagrama general de relaciones
4. Cada bloque explicado, entidad por entidad
5. El ciclo de vida del estado basal: borrador → versión firmada → histórico
6. Parte 2 — Propuesta de renombrado a español

1. Por qué llamamos "Baseline"

La palabra aparece en el código (clases, tablas, columnas) pero no en el lenguaje con el que hablamos del producto en español, donde siempre decimos "basal" o "estado basal". Hay dos motivos completamente distintos por los que la palabra "Baseline" aparece en el proyecto, y conviene no mezclarlos:

1. El "Baseline" clínico — el que importa para este documento.

Es la traducción, nunca hecha, de "estado basal" o "valoración basal" del residente: su situación funcional habitual — cómo se mueve, cómo come, si es continente, cómo se asea, cómo está su cognición, cómo se comunica, su conducta, su sueño y las ayudas que usa habitualmente, más el índice de Barthel (una escala clínica estándar de dependencia para las actividades básicas de la vida diaria). Este "estado basal" es la fotografía de referencia contra la que después se comparan los cambios del residente.

La aplicación se construyó primero como un prototipo en TypeScript, y en ese prototipo la carpeta y las clases de dominio ya se llamaban `baseline`. Al migrar ese prototipo a .NET, se tradujo la lógica pero se conservó literalmente el nombre de las clases y tablas — de ahí `BaselineModels.cs`, `baseline_drafts`, `BaselineArea`, etc. Lo curioso es que los *valores* de negocio dentro de esas clases sí que ya están en español desde el principio: el área de basal `Movilidad` se guarda en la base de datos con el código `MOVILIDAD`, el motivo de revisión `CAMBIO_FUNCIONAL_CONSOLIDADO`, etc. Es decir: el negocio siempre habló de "basal" en español; solo el "andamiaje" de código (nombres de clases, tablas y columnas) se quedó en inglés porque nadie lo tradujo al portar el prototipo.

2. El "Baseline" documental — no tiene nada que ver con el modelo de datos.

En la documentación del proyecto existe además el concepto de "Línea Base Funcional" (a veces abreviado LBF, y con nombres de fichero como `LBF-CONNECT-2026-09-06-V1.1`). Esto es una práctica de gestión de proyecto: cuando se aprueba una versión del documento de requisitos o de los bocetos de pantalla, se "congela" como referencia para no perder de vista qué se aprobó. Es la acepción de "baseline" que se usa en ingeniería de software en general (línea base de requisitos), y no tiene relación alguna con el residente ni con las tablas de base de datos. Se menciona aquí solo para que, si alguien encuentra la palabra "baseline" en un documento de producto, sepa que puede tratarse de esto y no de la valoración clínica.

En resumen: cuando en este documento (y en el código) aparece "Baseline" referido a tablas o clases, se refiere siempre al primer sentido — la valoración basal del residente. Es un nombre heredado de la traducción literal del prototipo original, no una decisión de diseño tomada para este proyecto en .NET.

2. Cómo está organizado el modelo: seis bloques funcionales

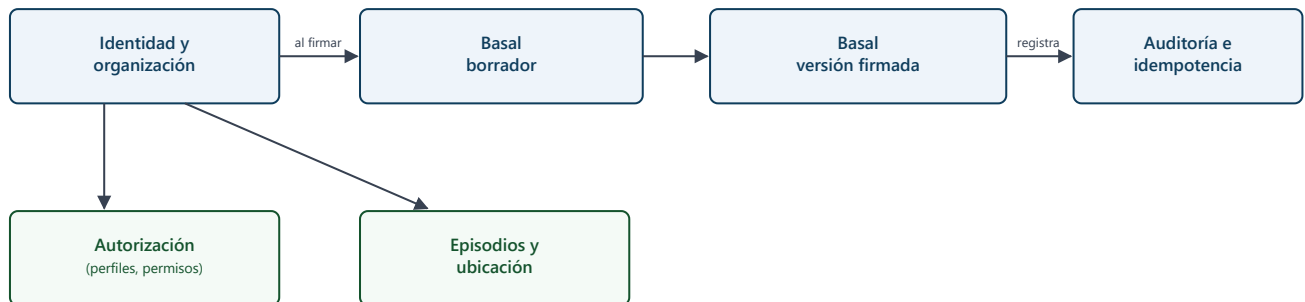
La base de datos actual (`database\scripts\0001_init_sqlserver.sql`) tiene 22 tablas. En vez de leerlas una a una sin orden, conviene verlas agrupadas en seis bloques, cada uno respondiendo a una pregunta distinta:

Bloque	Pregunta que responde	Tablas
Identidad y organización	¿Quién trabaja aquí, en qué centro, y quién es cada residente?	<code>accounts</code> , <code>centers</code> , <code>units</code> , <code>residents</code>
Autorización	¿Qué puede ver y hacer cada cuenta, con qué perfil, en qué centro/unidad/residente?	<code>profile_scopes</code> , <code>profile_unit_scopes</code> , <code>profile_resident_scopes</code> , <code>profile_permissions</code>
Episodios y ubicación	¿Desde cuándo está el residente en este centro, y dónde está ahora mismo dentro de él?	<code>resident_center_episodes</code> , <code>resident_location_intervals</code>
Basal — borrador	¿Qué se está valorando ahora mismo, todavía sin firmar?	<code>baseline_drafts</code> , <code>baseline_draft_areas</code> , <code>baseline_draft_barthel</code> , <code>baseline_draft_barthel_items</code>
Basal — versión firmada	¿Cuál fue (y es) la valoración basal oficial de cada residente, con quién la firmó y cuándo?	<code>baseline_versions</code> , <code>baseline_version_areas</code> , <code>baseline_version_barthel</code> , <code>baseline_version_barthel_items</code> , <code>resident_current_baselines</code> , <code>baseline_supersessions</code>
Auditoría e idempotencia	¿Quién hizo qué, cuándo, y cómo evitamos que una petición repetida duplique un cambio?	<code>audit_events</code> , <code>idempotency_operations</code>

Nota: `BaselineModels.cs` (el fichero que motivó la pregunta inicial) no es una de estas tablas — es un modelo de formulario web (MVC) que recoge los datos que llegan desde la pantalla para firmar un borrador o consultar el histórico. Vive en `src\ResidApp.Web\Models\` , una capa por encima de las tablas.

3. Diagrama general de relaciones

Este primer diagrama muestra cómo se conectan los seis bloques entre sí, sin entrar todavía en el detalle de cada tabla:



Toda la actividad parte de "Identidad y organización" (quién existe): de ahí cuelgan los permisos de cada cuenta, la ubicación del residente, y el propio ciclo de la valoración basal. La valoración basal nace como borrador editable, se convierte en versión firmada e inmutable, y cada acción relevante queda registrada en auditoría.

4. Cada bloque explicado, entidad por entidad

4.1 Identidad y organización

Tabla	Qué representa
<code>accounts</code>	Una cuenta de acceso (una persona que entra en la aplicación). No indica por sí sola qué puede hacer — eso lo dan los "ámbitos de perfil" del siguiente bloque.
<code>centers</code>	Una residencia/centro. Es la unidad organizativa de más alto nivel: casi todo lo demás cuelga de un centro.
<code>units</code>	Una unidad dentro de un centro (p. ej. una planta o módulo asistencial). Puede colgar de un edificio/planta, aunque esas entidades (edificios, plantas, habitaciones) todavía no se han portado del prototipo original.
<code>residents</code>	Una persona residente. Guarda su identidad básica y su estado (activo/inactivo), con quién y cuándo se le dio de baja si procede.

4.2 Autorización

Este bloque separa dos preguntas que se confunden fácilmente: "¿qué perfil tiene esta cuenta?" y "¿a qué residentes/unidades concretas puede acceder con ese perfil?". Por eso hay cuatro tablas en vez de una:

Tabla	Qué representa
<code>profile_scopes</code>	Que una cuenta tiene un perfil concreto (Enfermería, Medicina, Auxiliar, Familiar, Administración, Dirección Clínica) en un centro. Es el "carné" base.
<code>profile_unit_scopes</code>	Restringe ese carné a unidades concretas del centro (si no se restringe, se entiende que aplica a todo el centro).
<code>profile_resident_scopes</code>	Igual que la anterior pero restringiendo a residentes concretos, por ejemplo para un familiar que solo debe ver a su familiar.
<code>profile_permissions</code>	Qué acciones concretas habilita ese perfil (crear residente, completar basal inicial, reevaluar, leer detalle clínico...).

Estas cuatro tablas comparten una regla poco habitual pero deliberada: **nunca se borra un permiso o un perfil, solo se revoca** (columna `revoked_at` / `status = REVOKED`). La base de datos lo impone con disparadores (*triggers*) que bloquean el `DELETE` directamente. Esto es clave para poder responder después "¿qué podía ver esta persona el día X?" — algo que en un contexto asistencial hay que poder demostrar.

4.3 Episodios y ubicación

Tabla	Qué representa
<code>resident_center_episodes</code>	Un periodo de estancia del residente en un centro (desde cuándo, y hasta cuándo si ya terminó). Como mucho un episodio "abierto" (sin fecha de fin) por residente.
<code>resident_location_intervals</code>	Dónde está el residente dentro del centro en cada momento (unidad, y en el futuro habitación/plaza). También como mucho un intervalo abierto por residente: la ubicación "actual".

Ambas tablas siguen el mismo patrón que la autorización: no se borran filas, solo se "cierran" (se les pone fecha de fin) y se abre una nueva. Así queda un historial completo de por dónde ha pasado el residente, en vez de solo saber dónde está ahora.

4.4 y 4.5 — Basal (borrador y versión firmada)

Este es el bloque central del modelo, y se explica con detalle en la sección 5 porque su lógica (por qué hay dos "copias" del mismo contenido) no se entiende bien tabla por tabla, sino como un flujo completo.

4.6 Auditoría e idempotencia

Tabla	Qué representa
audit_events	Un registro de "quién hizo qué, sobre qué recurso, cuándo y con qué perfil activo". Es inmutable (no se puede editar ni borrar) por diseño: es la base para poder auditar el uso de datos clínicos sensibles.
idempotency_operations	Evita que una misma acción (p. ej. "firmar este borrador") se ejecute dos veces si la petición llega repetida por un reintento de red. Cada operación sensible viaja con un identificador único (<code>OperationId</code> — es justo el campo que aparece en <code>BaselineModels.cs</code>) y esta tabla recuerda si esa operación ya se completó.

5. El ciclo de vida del estado basal: borrador → versión firmada → histórico

La pregunta que de verdad explica el diseño de este bloque es: **¿por qué no hay una sola tabla "basal" con los datos del residente, que se va actualizando?** La respuesta es que una valoración clínica firmada no se puede "editar" después sin dejar rastro — igual que no se tacha una anotación en una historia clínica en papel, se añade una entrada nueva. Por eso el modelo separa claramente dos mundos:

Mundo 1 — el borrador (mutable, de trabajo).

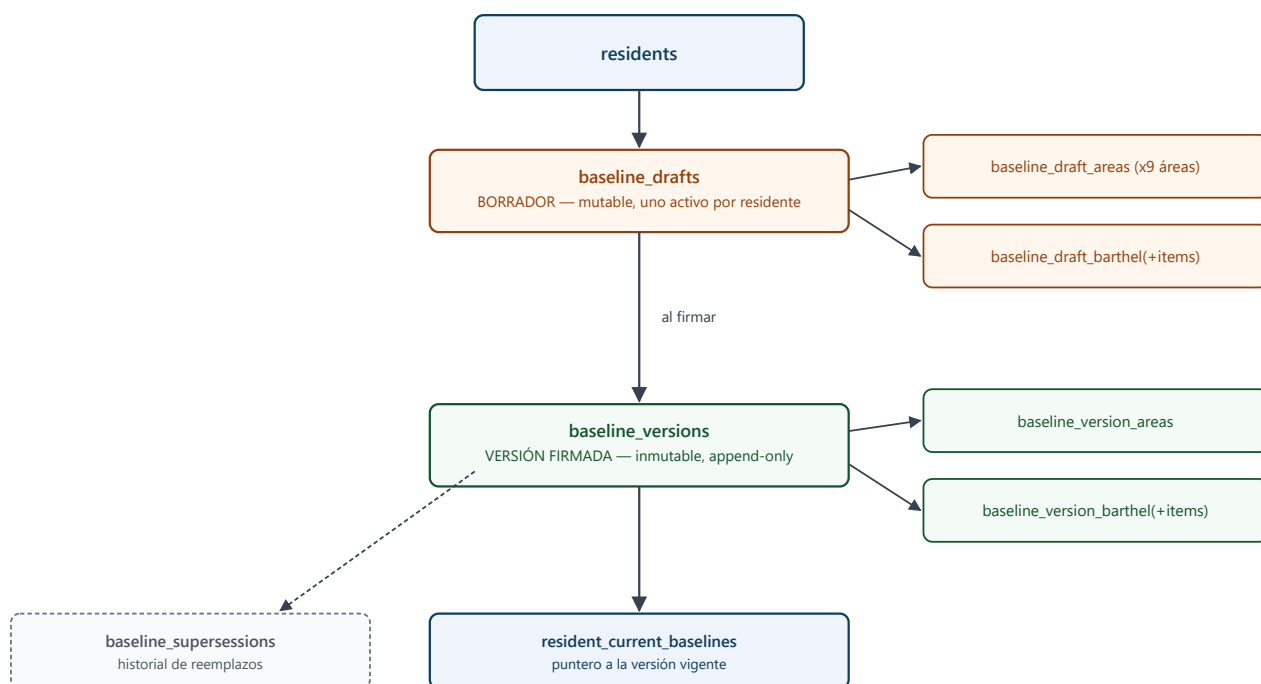
Mientras Enfermería o Medicina están rellenando las 9 áreas funcionales y el índice de Barthel, todo vive en `baseline_drafts` (la cabecera) y sus tablas hijas `baseline_draft_areas` y `baseline_draft_barthel` / `baseline_draft_barthel_items`. Aquí sí se puede corregir, porque todavía no es una valoración oficial. La base de datos solo permite **un borrador activo por residente a la vez** (para no tener dos personas rellenando basales distintos en paralelo sin saberlo), y ese límite está protegido a nivel de base de datos, no solo en el código de la aplicación.

Mundo 2 — la versión firmada (inmutable, para siempre).

Cuando se firma el borrador, su contenido se copia tal cual a `baseline_versions` y sus tablas hijas `baseline_version_areas` y `baseline_version_barthel` / `baseline_version_barthel_items`. A partir de ahí esas filas no se pueden ni actualizar ni borrar — la base de datos lo impide directamente (disparadores "append-only"). Cada firma añade una versión nueva (`version_number`), nunca sustituye a la anterior por encima: la anterior queda registrada como histórica.

Dos tablas más completan el ciclo:

- `resident_current_baselines` — un "puntero" con una sola fila por residente que dice cuál es la versión firmada *vigente ahora mismo*. Sin esta tabla, cada consulta tendría que calcular "la versión con el número más alto", lo cual es más lento y más fácil de hacer mal.
- `baseline_supersessions` — cuando una versión reemplaza a otra, aquí queda constancia de qué versión sustituyó a cuál y cuándo. Es lo que permite reconstruir la línea temporal completa de basales de un residente, no solo la última.



Flechas continuas: relación directa (clave foránea). Flecha discontinua: `baseline_supersessions` enlaza dos versiones firmadas entre sí (la anterior y la nueva), no nace directamente del residente.

Parte 2 — Propuesta de renombrado a español

Tablas y columnas, sin ñ, sin tildes ni diéresis. Es una propuesta de diseño para decidir; no se ha ejecutado ningún cambio en la base de datos ni en el código como parte de este documento.

Convención de nomenclatura

- `snake_case` en minúsculas, igual que ahora — no cambia el estilo, solo el idioma.
- Solo caracteres ASCII: las vocales pierden la tilde (`versión` → `version`) y la ñ se sustituye por `n` o `ni` según se lea mejor (`diseño` → `disenio` ; ningún nombre de este esquema usa hoy la ñ, así que no se ha necesitado aplicar esta regla en la práctica).
- Se reutiliza el mismo criterio que el propio dominio ya usa en sus valores de catálogo, que llevan años en español sin diacríticos: `MOVILIDAD` , `ALIMENTACION` , `SUENO` , `REVISION_PROGRAMADA` . Este documento extiende ese mismo criterio a los nombres estructurales (tabla/columna), que hasta ahora se habían quedado en inglés.
- Los *valores* guardados dentro de las columnas de código (`area_code` , `reason_code` , los perfiles, los ítems de Barthel...) ya están en español y no cambian — esta propuesta solo afecta a los nombres de tabla y columna, no al contenido.

Mapecto de tablas (22)

Tabla actual	Propuesta en español
<code>accounts</code>	<code>cuentas</code>

centers	centros
units	unidades
residents	residentes
profile_scopes	ambitos_perfil
profile_unit_scopes	ambitos_perfil_unidad
profile_resident_scopes	ambitos_perfil_residente
profile_permissions	permisos_perfil
resident_center_episodes	episodios_residente_centro
resident_location_intervals	intervalos_ubicacion_residente
baseline_drafts	basales_borrador
baseline_draft_areas	basales_borrador_areas
baseline_draft_barthel	basales_borrador_barthel
baseline_draft_barthel_items	basales_borrador_barthel_items
baseline_versions	basales_version
baseline_version_areas	basales_version_areas
baseline_version_barthel	basales_version_barthel
baseline_version_barthel_items	basales_version_barthel_items
resident_current_baselines	basales_vigentes_residente
baseline_supersessions	basales_sustituciones
audit_events	eventos_auditoria
idempotency_operations	operaciones_idempotencia

Mapeo de columnas, tabla por tabla

accounts → cuentas

Actual	Propuesta
id	id
external_subject	sujeto_externo
status	estado
created_at	creado_en

centers → centros

Actual	Propuesta
id	id
code	codigo
display_name	nombre_visible
status	estado
created_at	creado_en

units → unidades

Actual	Propuesta
id	id
center_id	centro_id
building_id	edificio_id
floor_id	planta_id
code	codigo
display_name	nombre_visible
status	estado
created_at	creado_en

residents → residentes

Actual	Propuesta
id	id
center_id	centro_id
display_name	nombre_visible
birth_date	fecha_nacimiento
documented_sex_code	sexo_documentado_codigo
status	estado
inactivation_reason	motivo_inactivacion
inactivated_at	inactivado_en
inactivated_by_account_id	inactivado_por_cuenta_id
inactivated_by_profile	inactivado_por_perfil
created_at	creado_en
created_by_account_id	creado_por_cuenta_id
created_by_profile	creado_por_perfil

profile_scopes → ambitos_perfil

Actual	Propuesta
id	id
account_id	cuenta_id
center_id	centro_id
profile_code	perfil_codigo
status	estado
granted_at	concedido_en
granted_by_account_id	concedido_por_cuenta_id
revoked_at	revocado_en
revoked_by_account_id	revocado_por_cuenta_id

profile_unit_scopes → ambitos_perfil_unidad

Actual	Propuesta
id	id
profile_scope_id	ambito_perfil_id
center_id	centro_id
unit_id	unidad_id
granted_at	concedido_en
granted_by_account_id	concedido_por_cuenta_id
revoked_at	revocado_en
revoked_by_account_id	revocado_por_cuenta_id

profile_resident_scopes → ambitos_perfil_residente

Actual	Propuesta
id	id
profile_scope_id	ambito_perfil_id
center_id	centro_id
resident_id	residente_id
granted_at	concedido_en
granted_by_account_id	concedido_por_cuenta_id
revoked_at	revocado_en
revoked_by_account_id	revocado_por_cuenta_id

profile_permissions → permisos_perfil

Actual	Propuesta
id	id
profile_scope_id	ambito_perfil_id
center_id	centro_id
permission_code	permiso_codigo
granted_at	concedido_en
granted_by_account_id	concedido_por_cuenta_id
revoked_at	revocado_en
revoked_by_account_id	revocado_por_cuenta_id

resident_center_episodes → episodios_residente_centro

Actual	Propuesta
id	id
resident_id	residente_id
center_id	centro_id
internal_reference	referencia_interna
valid_from	vigente_desde
valid_until	vigente_hasta
created_at	creado_en
created_by_account_id	creado_por_cuenta_id
created_by_profile	creado_por_perfil

resident_location_intervals → intervalos_ubicacion_residente

Actual	Propuesta
id	id
resident_id	residente_id
center_id	centro_id
episode_id	episodio_id
unit_id	unidad_id
building_id	edificio_id
floor_id	planta_id
room_id	habitacion_id
place_id	plaza_id
valid_from	vigente_desde
valid_until	vigente_hasta
changed_at	modificado_en
changed_by_account_id	modificado_por_cuenta_id
changed_by_profile	modificado_por_perfil

baseline_drafts → basales_borrador

Actual	Propuesta
id	id
resident_id	residente_id
center_id	centro_id
created_in_unit_id	creado_en_unidad_id
status	estado
reason_code	motivo_codigo
common_information_source_code	fuentes_informacion_comun_codigo
common_information_source_other_text	fuentes_informacion_comun_otro_texto
common_information_date	fecha_informacion_comun
created_by_account_id	creado_por_cuenta_id
created_by_profile	creado_por_perfil
created_at	creado_en
updated_by_account_id	actualizado_por_cuenta_id
updated_by_profile	actualizado_por_perfil
updated_at	actualizado_en
draft_revision	revision_borrador
cancelled_at	cancelado_en
cancelled_by_account_id	cancelado_por_cuenta_id
cancelled_by_profile	cancelado_por_perfil
cancellation_reason	motivo_cancelacion

baseline_draft_areas → basales_borrador_areas

Actual	Propuesta
id	id
draft_id	borrador_id
resident_id	residente_id
center_id	centro_id
area_code	area_codigo
catalog_version_code	catalogo_version_codigo
answer_payload	respuestas_json
observation	observacion
information_source_override_code	fuentes_informacion_sustituta_codigo
information_source_override_other_text	fuentes_informacion_sustituta_otro_texto
information_date_override	fecha_informacion_sustituta
recorded_by_account_id	registrado_por_cuenta_id
recorded_by_profile	registrado_por_perfil
recorded_at	registrado_en

baseline_draft_barthel → basales_borrador_barthel

Actual	Propuesta
id	id
draft_id	borrador_id
resident_id	residente_id
center_id	centro_id
instrument_version_code	instrumento_version_codigo
assessment_date	fecha_valoracion
total_score	puntuacion_total
recorded_by_account_id	registrado_por_cuenta_id
recorded_by_profile	registrado_por_perfil
recorded_at	registrado_en

baseline_draft_barthel_items → basales_borrador_barthel_items

Actual	Propuesta
id	id
barthel_id	barthel_id
draft_id	borrador_id
resident_id	residente_id
center_id	centro_id
instrument_version_code	instrumento_version_codigo
item_code	item_codigo
selected_option_code	opcion_seleccionada_codigo
awarded_score	puntuacion_otorgada

baseline_versions → basales_version

Actual	Propuesta
id	id
source_draft_id	borrador_origen_id
resident_id	residente_id
center_id	centro_id
created_in_unit_id	creado_en_unidad_id
version_number	numero_version
reason_code	motivo_codigo
common_information_source_code	fuentes_informacion_comun_codigo
common_information_source_other_text	fuentes_informacion_comun_otro_texto
common_information_date	fecha_informacion_comun
created_by_account_id	creado_por_cuenta_id
created_by_profile	creado_por_perfil
created_at	creado_en
signed_by_account_id	firmado_por_cuenta_id
signed_by_profile	firmado_por_perfil
signed_at	firmado_en
valid_from	vigente_desde
activation_operation_id	operacion_activacion_id

baseline_version_areas → basales_version_areas

Actual	Propuesta
id	id
baseline_version_id	version_basal_id
resident_id	residente_id
center_id	centro_id
area_code	area_codigo
catalog_version_code	catalogo_version_codigo
answer_payload	respuestas_json
observation	observacion
information_source_override_code	fuentes_informacion_sustituta_codigo
information_source_override_other_text	fuentes_informacion_sustituta_otro_texto
information_date_override	fecha_informacion_sustituta
recorded_by_account_id	registrado_por_cuenta_id
recorded_by_profile	registrado_por_perfil
recorded_at	registrado_en

baseline_version_barthel → basales_version_barthel

Actual	Propuesta
id	id
baseline_version_id	version_basal_id
resident_id	residente_id
center_id	centro_id
instrument_version_code	instrumento_version_codigo
assessment_date	fecha_valoracion
total_score	puntuacion_total
recorded_by_account_id	registrado_por_cuenta_id
recorded_by_profile	registrado_por_perfil
recorded_at	registrado_en

baseline_version_barthel_items → basales_version_barthel_items

Actual	Propuesta
id	id
barthel_id	barthel_id
baseline_version_id	version_basal_id
resident_id	residente_id
center_id	centro_id
instrument_version_code	instrumento_version_codigo
item_code	item_codigo
selected_option_code	opcion_seleccionada_codigo
awarded_score	puntuacion_otorgada

resident_current_baselines → basales_vigentes_residente

Actual	Propuesta
resident_id	residente_id
center_id	centro_id
baseline_version_id	version_basal_id
activated_at	activado_en

baseline_supersessions → basales_sustituciones

Actual	Propuesta
previous_version_id	version_anterior_id
new_version_id	version_nueva_id
resident_id	residente_id
center_id	centro_id
superseded_at	sustituido_en

audit_events → eventos_auditoria

Actual	Propuesta
id	id
account_id	cuenta_id
active_profile	perfil_activo
center_id	centro_id
unit_id	unidad_id
resident_id	residente_id
resource_type	tipo_recurso
resource_id	recurso_id
action_code	accion_codigo
purpose_code	proposito_codigo
occurred_at	ocurrido_en

`idempotency_operations` → `operaciones_idempotencia`

Actual	Propuesta
<code>id</code>	<code>id</code>
<code>account_id</code>	<code>cuenta_id</code>
<code>action_code</code>	<code>accion_codigo</code>
<code>operation_id</code>	<code>operacion_id</code>
<code>request_hash</code>	<code>hash_solicitud</code>
<code>status</code>	<code>estado</code>
<code>result_resource_id</code>	<code>recurso_resultado_id</code>
<code>result_json</code>	<code>resultado_json</code>
<code>created_at</code>	<code>creado_en</code>
<code>completed_at</code>	<code>completado_en</code>

Qué haría falta para aplicar esta propuesta

Como el proyecto no usa Entity Framework Core sino Dapper con SQL directo (un único script `database\scripts\0001_init_sqlserver.sql`, sin carpeta de migraciones EF), aplicar este renombrado no sería "generar una migración" en el sentido de Entity Framework, sino:

1. Escribir un nuevo script T-SQL que renombre tablas y columnas (con `sp_rename` si hay que conservar datos, o recreando el esquema si no los hay).
2. Renombrar en paralelo las clases, propiedades y consultas SQL equivalentes en `ResidApp.Domain.Baseline`, `ResidApp.Application`, `src\ResidApp.Infrastructure\Persistence\SqlBaselineRepository.cs` y las vistas `Views\Baseline\*` — del orden de 30 archivos, a día de hoy con pruebas en verde sobre los nombres actuales.

Es, por tanto, un cambio amplio que conviene planificar y aprobar como un trabajo aparte, una vez que esta propuesta de nombres esté validada.